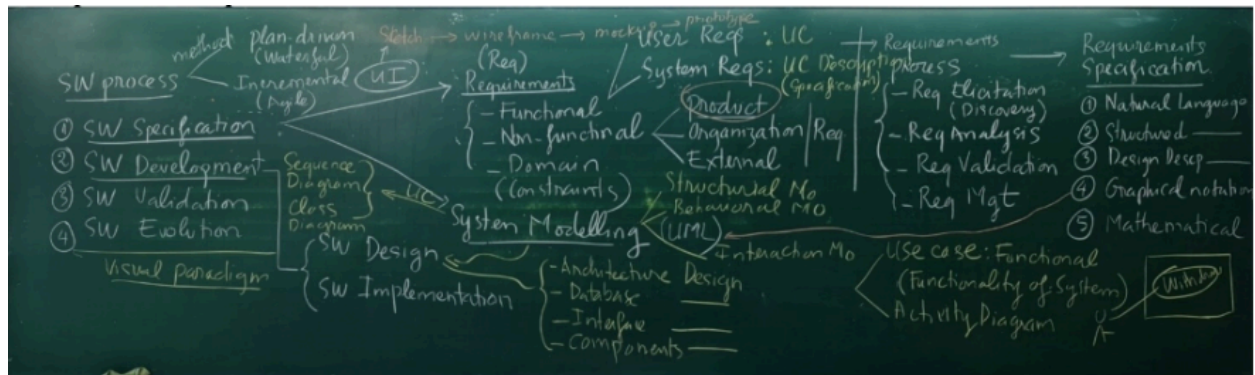


Hint from picture extract



This is a fantastic chalkboard summary of core Software Engineering principles, perfectly laid out for a midterm review! It maps out the entire lifecycle of building software, from figuring out what to build (requirements) to how to build it (design and modeling).

Here is a breakdown of the concepts presented on the board, followed by a text-based mindmap you can use to structure your studying.

1. The Software (SW) Process

The left side of the board outlines the fundamental phases of creating software.

- **The 4 Core Activities:**

1. **SW Specification:** Defining *what* the system should do and its constraints.

This is the process of establishing the services a customer requires from a system and the constraints under which it operates and is developed. It is an iterative process known as requirements engineering, which includes elicitation (discovering what stakeholders want), specification (documenting requirements in detail), and validation (checking that the requirements truly support the customer's needs)

2. **SW Development:** Designing and programming the software.

This stage realizes the requirements by transforming a specification into an executable software system. It involves two interleaved sub-activities:

- **Software Design:** A creative process where you identify software components (architectural, database, interface, and component design) and their relationships.
- **Implementation:** The process of translating the design into a program. Modern development focuses heavily on software reuse, configuration management to track changes, and host-target development

3. **SW Validation:** Testing the software to ensure it does what the customer wants.

*Validation is intended to show that a system conforms to its specification and meets the real needs of the customer. While it includes review processes, **testing** is the most common activity. It typically proceeds through several stages: **development testing** (unit, component, and system testing), **release testing** (checking a complete version for external use), and **user testing** (alpha, beta, or acceptance testing)*

4. **SW Evolution:** Modifying the software to adapt to changing customer and market needs.

*Software is inherently flexible and must evolve as business circumstances and requirements change. Evolution includes **software maintenance**, which involves repairing faults, adapting the system to new environments, and adding new functionality. For older **legacy systems**, organizations must decide whether to scrap, maintain, reengineer, or replace them based on their business value and quality*

- **Methodologies:** The board contrasts **Plan-driven (Waterfall)**, where phases flow sequentially downward, with **Incremental (Agile)**, where development happens in iterative cycles.

2. Requirements Engineering (Req)

This takes up the bulk of the upper-middle and right sections. Requirements are the foundation of any project.

- **Types of Requirements:**
 - **Functional:** What the system actually does (e.g., a user can log in).
 - **Non-functional:** How well the system performs its functions. The board breaks this into **Product** (e.g., performance, security), **Organization** (e.g., company policies), and **External** (e.g., legal, regulatory).
 - **Domain:** Constraints dictated by the specific industry the software is built for.
- **The Requirements Process:**
 - **Elicitation (Discovery):** Gathering what users need.
 - **Analysis:** Organizing and understanding those needs.
 - **Validation:** Checking that the requirements are correct and complete.
 - **Management:** Handling changes to requirements as the project progresses.
- **Requirements Specification (How to write them down):**
 1. Natural Language (plain English)

2. Structured Language (formatted text/templates)
3. Design Description (pseudocode)
4. Graphical Notation (diagrams)
5. Mathematical (formal specifications)

3. System Modeling (UML)

The middle-bottom section focuses on visualizing the system before coding, heavily relying on UML (Unified Modeling Language).

- **Types of Models:**
 - **Structural Models:** How the system is organized (e.g., Class Diagrams).
 - **Behavioral Models:** What happens in the system (e.g., Activity Diagrams).
 - **Interaction Models:** How components talk to each other (e.g., Sequence Diagrams).
- **Use Cases (UC):** The board highlights Use Cases as a way to capture functional requirements from the user's perspective. There is a classic diagram sketched on the bottom right: an "Actor" (the stick figure) interacting with a system boundary to perform a function (like "Withdraw" from an ATM).

4. Software Design & UI

The bottom left branches from "SW Development" into design specifics.

- **SW Design Sub-components:**
 - Architecture Design (the overall structure)
 - Database (data storage)
 - Interface (how systems/users interact)
 - Components (individual functional modules)
- **User Interface (UI) Evolution:** At the very top, there is a helpful progression for designing interfaces: **Sketch -> Wireframe -> Mockup -> Prototype**.
- **Tools:** "Visual Paradigm" is jotted down, which is a popular software tool used for drawing UML diagrams.

Your Midterm Review Mindmap

You can copy this text-based mindmap into a note-taking app (like Obsidian, Notion, or Roam) to create flashcards or expand on definitions!

- **Software Engineering Core Concepts**
 - **Software Process Lifecycle**

1. SW Specification (What to build)
2. SW Development (How to build it)
3. SW Validation (Testing it)
4. SW Evolution (Maintaining/Updating it)

- **Process Methodologies**

- Plan-driven (Waterfall)
- Incremental (Agile)

- **UI Design Progression**

- Sketch -> Wireframe -> Mockup -> Prototype

- **Requirements Engineering**

- **Types of Requirements**

- Functional (System capabilities)
- Non-functional (Quality attributes)
 - Product
 - Organization
 - External
- Domain (Industry constraints)

- **Requirements Process**

- Elicitation (Discovery)
- Analysis
- Validation
- Management

- **Specification Methods**

1. Natural Language
2. Structured Language

3. Design Description

4. Graphical Notation

5. Mathematical

- **System Modelling (UML)**

- Structural Models (e.g., Class Diagram)
- Behavioral Models (e.g., Activity Diagram)
- Interaction Models (e.g., Sequence Diagram)
- Use Cases (Actor + Functionality, e.g., "Withdraw")

- **Software Design (Translating Reqs to Dev)**

- Architecture Design
- Database Design
- Interface Design
- Component Design